

A Full-GPU PH-SHOWOA with SA-RCRS-GRASP Initialization for VRPSPDTW

Huynh Khang Lam³, Anh Huy Bui³, Tri Minh Pham^{1,2,3*}

^{1*}Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam.

²Vietnam National University, Ho Chi Minh City, Ho Chi Minh City, Vietnam.

³AiTA Lab, Faculty of Information Technology, FPT University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): tri.phamhpc@hcmut.edu.vn;
Contributing authors: khanglhse200666@fpt.edu.vn; huy@fpt.edu.vn;

Abstract

The Vehicle Routing Problem with Simultaneous Pickup and Delivery and Time Windows (VRPSPDTW) is difficult because every route must satisfy capacity and time-window constraints while the number of vehicles is minimized before travel distance. PH-SHOWOA is a parallel hybrid of the Spotted Hyena Optimizer (SHO) and Whale Optimization Algorithm (WOA). Its original implementation was designed for CPU execution. This paper presents fullGPU-SA-RCRS-GRASP, a Python CUDA implementation that keeps the solution population, route data, objective values, random states, and search operators on the GPU during a run. The original simulated-annealing initialization is replaced by an SA-RCRS-GRASP construction that uses a restricted candidate list and a short simulated-annealing warm-up. The implementation uses fixed-shape tensors and preallocated device buffers, so it avoids CPU-GPU synchronization inside the search loop. On 15 Wang-Chen benchmark instances, the GPU implementation preserves the number of vehicles on 14 instances and improves it on one instance. It obtains equal or shorter total distance on 14 instances. The aggregate runtime falls from 104,175 s to 3,648.2 s, which is a 28.6-fold speedup for the reported experiment. These results show that a GPU-resident design can greatly reduce runtime while maintaining competitive routing quality.

Keywords: VRPSPDTW, GPU computing, metaheuristic, PH-SHOWOA, SA-RCRS-GRASP, CUDA

1 Introduction

The vehicle routing problem with simultaneous pickup and delivery and time windows (VRPSPDTW) asks for a set of routes that serve all customers. A vehicle may deliver goods and collect goods at the same customer, and every service must respect its time window and the vehicle capacity. Following the original PH-SHOWOA paper, reducing the number of vehicles (NV) is the primary objective because it has a large fixed operating cost. Total distance (TD) is compared only after NV.

The VRPSPDTW extends classical time-window routing [1, 2] by requiring simultaneous delivery and pickup. Its model, practical instances, exact methods, and multiobjective variants have been studied in several forms [3–5]. The VRPSPDTW has also been studied with genetic algorithms, parallel simulated annealing, adaptive large-neighborhood search, lexicographic two-stage search, and memetic search [6–11]. These studies support the use of a hierarchical objective: minimize NV first and then TD [7, 9]. Recent studies show the value of local search, hybridization, and multi-stage search when pickup, delivery, capacity, and time windows interact [12–16]. PH-SHOWOA combines the exploration behaviour of the Spotted Hyena Optimizer (SHO) with the best-guided intensification of the Whale Optimization Algorithm (WOA) [17–22]. The original work also uses simulated annealing, local search, diversification, and parallel solution updates. Although these components produce good routes, route construction, feasibility checks, and population updates create many small computations. A conventional CPU-oriented implementation cannot fully use the large parallel capacity of a modern GPU.

GPU computing provides a natural way to evaluate and update many population members at once. CUDA was designed for scalable parallel execution of data-parallel workloads [23], while recent tensorized evolutionary optimization shows that candidate populations, objective values, and variation operators can be reformulated as dense device tensors [24]. GPU acceleration has also been investigated for metaheuristic benchmarking and vehicle-routing local optimization [25, 26]. Parallel portfolio construction gives another useful view of distributing diverse search behavior across concurrent workers [27]. However, accelerating only route evaluation still leaves repeated host-device transfers and CPU-side search control as sources of overhead. This work develops a full-GPU version of PH-SHOWOA in Python. The proposed method changes the initial construction to SA-RCRS-GRASP and performs the full search of one run on the GPU. CPU work is limited to preparing immutable instance data and decoding the final best solution after all runs. The main aim is to reduce runtime without changing the NV-first priority of the original method.

The research gap is therefore not only a faster route evaluator, but a complete PH-SHOWOA search procedure that stays on the GPU. The present work addresses this gap through a GPU-resident population representation, a GPU implementation of SA-RCRS-GRASP initialization, and device-side search and repair operators. The evaluation examines whether this design reduces runtime while preserving the original NV-first routing objective. The source code, benchmark inputs, and scripts are publicly available at <https://github.com/Welkie/ph-showoa>.

2 Materials and methods

2.1 Datasets

The study uses Wang–Chen VRPSPDTW benchmark instances recorded in the project repository [6]. They include random (Rdp), clustered (Cdp), and mixed random–clustered (RCdp) customer layouts. Each instance specifies customer delivery and pickup demand, service time, capacity, time windows, and distance and travel-time matrices. The reported experiment contains 15 instances: one instance with 10 customers, three with 50 customers, and eleven with 100 customers. These varied layouts make it possible to examine both small and medium-size routing cases.

2.2 Problem formulation and baseline framework

Let $V = \{0, 1, \dots, n\}$ be the depot and n customers. Each customer has delivery demand, pickup demand, service time, and a time window. A feasible route begins and ends at the depot, visits each customer exactly once over all routes, and respects the time and capacity constraints at every route position. The solution quality is compared lexicographically: a solution with fewer vehicles is always preferred; only when two solutions use the same number of vehicles is the shorter TD preferred. For implementation, the same preference is represented by the scalar score

$$F(S) = 2000 NV(S) + TD(S), \quad (1)$$

where the large vehicle weight makes the search prefer a vehicle reduction before a distance reduction.

The baseline PH-SHOWOA alternates SHO-like population diversification and WOA-like intensification around the current best solution [22]. It also applies simulated-annealing acceptance and periodic route improvements. This work retains this search principle, but redesigns the data and operators for a GPU-resident execution.

2.3 Proposed full-GPU PH-SHOWOA method

Figure 1 gives the algorithmic flow of the proposed PH-SHOWOA. It retains the SHO/WOA hybrid search of the original method, but replaces the original initial-population step with SA-RCRS-GRASP. Every acceptance and best-solution comparison follows the NV-first, TD-second rule. The flow is algorithmic; Figure 2 separately shows where CPU and GPU execution occur.

2.3.1 GPU-resident tensor representation

Each population member is stored as fixed-size route tensors. The node tensor stores customer indices for every route position, while companion tensors store route lengths, number of routes, total distance, and fitness. Demand vectors, time-window vectors, service times, and distance and travel-time matrices are copied to the device once before the runs start. The implementation also allocates current-population,

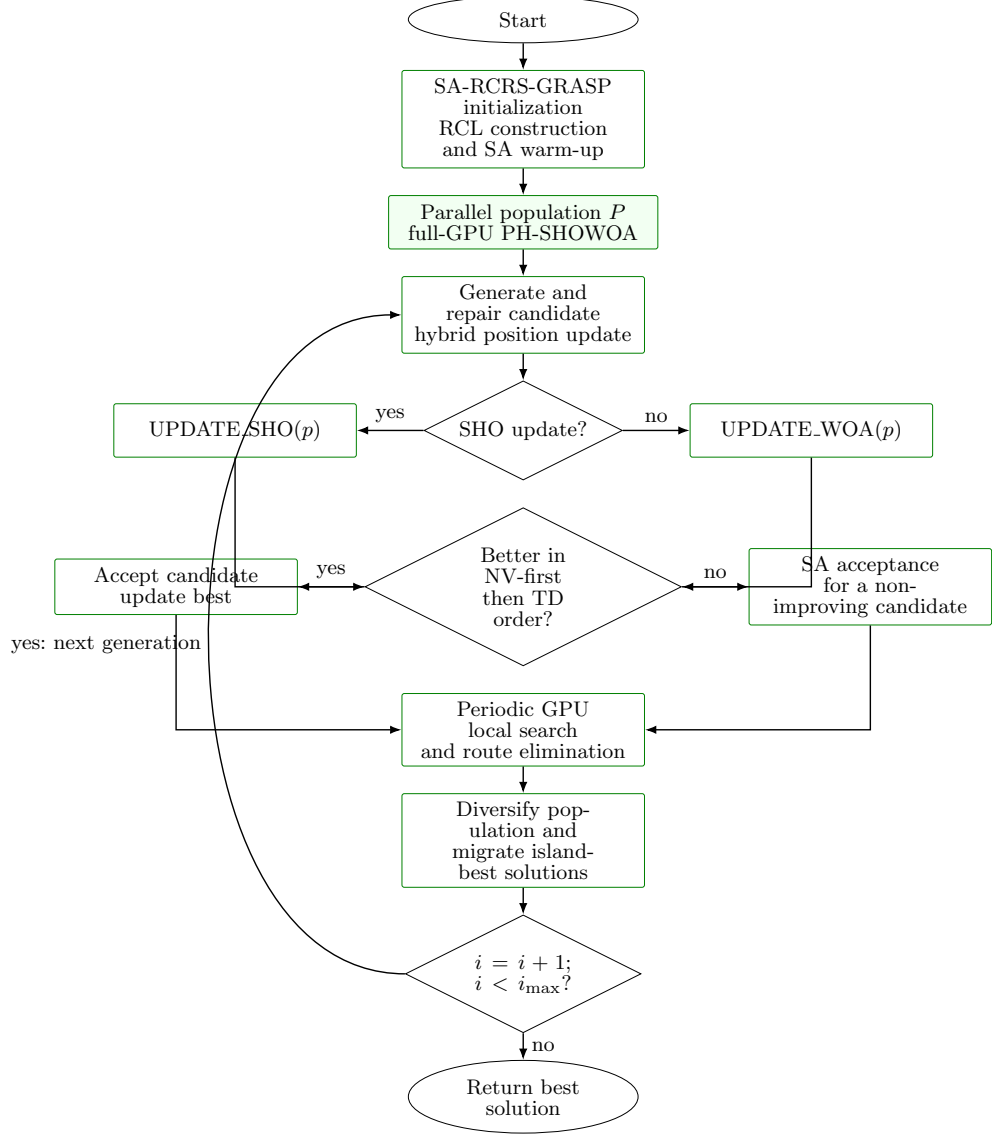


Fig. 1 The flow chart of the proposed Parallel Hybrid based on Spotted Hyena Optimizer and Whale Optimization Algorithm (PH-SHOWOA), with SA-RCRS-GRASP initialization. Candidate and best-solution comparisons use NV first and TD second.

next-population, candidate, island-best, global-best, random-state, and scratch tensors before the search. This layout follows the central idea of tensorization: express a population and its related values in dense arrays so that many candidates can be processed at the same time [24].

For each run, a deterministic seed creates one random state per population member. Initialization, objective evaluation, feasibility repair, SHO/WOA update, local search,

diversification, island migration, and best-solution tracking are launched as CUDA kernels. The current and next populations use a ping-pong swap, so no host copy is needed between generations. At the end of all runs, only the final best tensor is copied back and decoded into the ordinary route representation. This design removes CPU–GPU synchronization from the inner optimization loop.

2.3.2 SA-RCRS-GRASP initialization

The initial population must be diverse but feasible enough for later SHO/WOA updates. The proposed initializer first constructs routes using a route-customer relation score (RCRS). For each unserved customer, feasible insertions are evaluated by their route effect, including added distance, route compatibility, and time-window feasibility. Instead of always selecting the single best insertion, GRASP forms a restricted candidate list from good feasible insertions. A random choice from this list introduces controlled diversity. The GRASP parameter α is sampled within the configured interval $[0.10, 0.40]$ for different population members.

After construction, a short simulated-annealing warm-up improves the route. A candidate move that improves the NV-first objective is accepted. A worse move may also be accepted with probability

$$P(\text{accept}) = \exp\left(-\frac{F(S') - F(S)}{T}\right), \quad (2)$$

where T is the current temperature. This rule, introduced by simulated annealing [28], helps the initializer escape a weak greedy choice. All RCRS scores, restricted-list decisions, SA moves, and random draws are computed on the GPU. The resulting population therefore starts with both structure and diversity without a CPU construction phase. GRASP is a well-known method for combining greedy construction and randomized selection [29].

2.3.3 GPU search and improvement

At generation t , the method uses the same hybrid search idea as PH-SHOWOA. The WOA control value decreases from 2 to 0, while the probability of an SHO-style update decreases smoothly during the search. Early generations therefore emphasize diversified moves, whereas later generations put more emphasis on the best solution. Each candidate is repaired and evaluated on the device before replacement. Periodically, GPU kernels perform route elimination and local moves, including relocation, swap, and route-order improvement. If an island stops improving, a diversification kernel changes a subset of its population. Island-best solutions can migrate at a fixed interval, which spreads useful information without forcing all members to become identical.

Figure 2 summarizes the full-GPU workflow. The key visual message is that the arrows inside the run remain inside GPU memory.

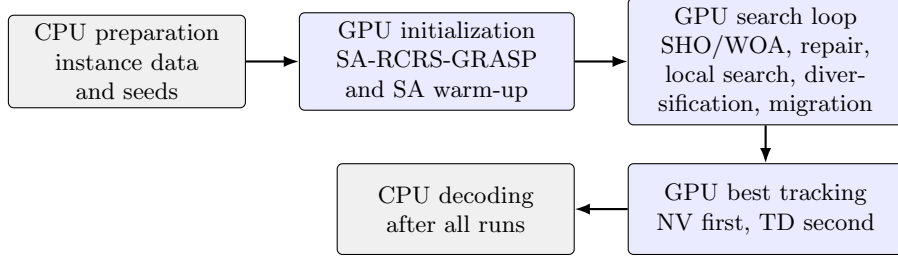


Fig. 2 Full-GPU PH-SHOWOA workflow. The GPU search and best-tracking kernels repeat for each generation; the search state remains on the GPU during a run. CPU-GPU transfer occurs only for preparation and final decoding.

3 Results

3.1 Experimental configurations and evaluation metrics

The comparison uses the Python PH-SHOWOA implementation as the reference and the fullGPU-SA-RCRS-GRASP Python implementation as the proposed method. For each instance, we report NV, TD, and average runtime per run. The stored results should be accompanied in the final submission by the exact GPU model, CPU model, RAM, CUDA/Numba versions, population size, iteration limit, number of runs, and random-seed policy. These details are necessary for a reproducible runtime comparison.

The reported quality values are compared in the original NV-first order: NV first and TD second. Runtime speedup is calculated as $t_{\text{PH-SHOWOA}}/t_{\text{GPU}}$. Because the two implementations use randomized search, future experiments should run both methods with the same seed set and report mean and standard deviation or confidence intervals. The current table is therefore an empirical performance record, not a statistical significance claim.

3.2 Model performance analysis

Table 1 presents the supplied results. The full-GPU method uses the same NV as the reference on 14 of 15 instances and reduces NV from 14 to 13 on Rdp103. It obtains equal or lower TD on 14 instances; Rdp103 is the only case with a larger TD because the proposed method uses one fewer vehicle. Under the NV-first objective of the original paper, this is an improvement because the vehicle reduction takes precedence over the distance increase.

The runtime difference is large across all instance sizes. The total of the reported average runtimes is 104,175 s for the reference and 3,648.2 s for fullGPU-SA-RCRS-GRASP, giving an aggregate speedup of $28.6\times$. Instance speedups range from $20.3\times$ to $105.3\times$, with a median of $30.8\times$. The gain is strongest on the 10-customer instance, while the 100-customer instances still show substantial reductions. The aggregate TD is 15,447.297 for the proposed method and 15,810.510 for the reference, a 2.30% reduction on this selected set. These results support the claim that keeping the complete search state on the GPU removes important overhead without an obvious loss of routing quality.

Table 1 Reported comparison between fullGPU-SA-RCRS-GRASP and the Python PH-SHOWOA reference. Lower NV, TD, and time are better.

Instance	Size	fullGPU-SA-RCRS-GRASP			PH-SHOWOA reference			Speedup ($\times$)
		NV	TD	Avg./run (s)	NV	TD	Avg./run (s)	
RCdp1001	10	3	348.980	1.70	3	348.980	179	105.3
RCdp5001	50	9	994.180	27.90	9	996.350	1316	47.2
RCdp5007	50	7	811.080	29.73	7	816.880	1424	47.9
RCdp5004	50	6	733.200	31.07	6	747.470	1479	47.6
RCdp101	100	15	1670.310	121.17	15	1707.340	3229	26.6
Cdp103	100	10	932.777	121.07	10	939.220	4780	39.5
Rcdp205	100	4	1332.860	467.20	4	1427.400	10730	23.0
Rdp210	100	3	959.980	584.60	3	1033.360	17990	30.8
Rcdp207	100	3	1108.940	483.76	3	1133.680	14114	29.2
Rcdp202	100	4	1176.990	584.00	4	1282.590	11884	20.3
Rdp103	100	13	1382.690	152.20	14	1262.560	4084	26.8
Cdp104	100	10	897.060	120.27	10	898.900	4679	38.9
Cdp102	100	10	959.470	116.90	10	997.620	4439	38.0
Rdp203	100	3	947.060	662.90	3	1006.190	19546	29.5
Rcdp104	100	11	1191.720	143.73	11	1211.970	4302	29.9

4 Discussion

4.1 Component interpretation and ablation need

The observed runtime reduction is consistent with the design of the proposed method. Preallocated device tensors avoid repeated allocation, and the GPU-resident loop avoids copying the population between CPU and GPU at every generation. The SA-RCRS-GRASP initializer adds structured and randomized feasible constructions before the hybrid search begins. However, the current results do not isolate the contribution of each component. A future ablation study should compare the CPU reference, GPU residency alone, GPU residency with RCRS-GRASP, and the complete SA-RCRS-GRASP method under identical seeds and stopping conditions.

4.2 Comparison with the reference implementation

Relative to the Python PH-SHOWOA reference, the proposed method retains or improves NV on all 15 reported instances. It also improves or matches TD on 14 instances. The Rdp103 result illustrates the importance of using the original comparison rule: the proposed method uses one fewer vehicle, so its larger TD does not make it worse under the NV-first objective. The result table therefore supports the intended quality priority rather than a distance-only comparison.

4.3 Practical implications

The full-GPU design is useful when many independent population members and route operations must be processed repeatedly. Its main practical benefit is shorter wall-clock time, which can allow more runs, larger populations, or additional parameter studies

within a fixed computing budget. The GPU workflow is also suitable for experiments where solutions should remain on the device until a final answer is required.

4.4 Limitations and future prospects

The present method has limitations that come from its algorithmic scope. First, its fixed-shape route and scratch tensors require preselected bounds for the number of routes and route positions. This design is efficient for the tested instances, but it uses memory proportional to population size and problem size. Very large instances may require a smaller population, a compact encoding, or a different kernel layout. Second, SA-RCRS-GRASP, local-search intervals, stagnation control, and island migration introduce hyperparameters. Their current values are effective for the reported configuration, but their best settings can depend on customer distribution, time-window tightness, and GPU memory capacity. Adaptive parameter control is therefore a useful next step.

The implementation also addresses a static, single-depot VRPSPDTW with deterministic travel times and a homogeneous fleet. It does not yet model real-time requests, uncertain travel times, multiple depots, heterogeneous vehicles, driver regulations, or explicit emissions. Extending the GPU kernels to these operational features is not a simple data-format change because every feasibility and repair operator must be adapted. Future work can evaluate compact tensor encodings, adaptive search control, and dynamic or stochastic extensions while preserving the NV-first decision rule.

5 Conclusion

This paper presented fullGPU-SA-RCRS-GRASP, a Python CUDA implementation of PH-SHOWOA for VRPSPDTW. The method stores the population and search state as preallocated tensors and performs initialization, SA warm-up, SHO/WOA evolution, route improvement, diversification, migration, and best tracking on the GPU. SA-RCRS-GRASP replaces the original initialization with a construction that balances greediness and diversity. On the supplied 15-instance result set, the method retains or improves NV on every instance, obtains equal or lower TD on 14 instances, and reduces the aggregate reported runtime by $28.6\times$. Future work will test the complete benchmark under matched seeds and hardware, add statistical tests, and further improve GPU route operators for harder 100-customer instances.

Declarations

Data and code availability

The implementation, benchmark inputs, and experiment scripts are available at <https://github.com/Welkie/ph-showoa>. The full-GPU Python implementation is in `src.python_gpu_SA_RCRS_GRASP`. The original PH-SHOWOA study reports that its underlying benchmark data are available from the VRPenstein repository [22].

Competing interests

The authors declare no competing interests.

Funding

No external funding was received for this study.

References

- [1] Solomon, M.M.: Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research* **35**(2), 254–265 (1987) <https://doi.org/10.1287/opre.35.2.254>
- [2] Laporte, G.: Fifty years of vehicle routing. *Transportation Science* **43**(4), 408–416 (2009) <https://doi.org/10.1287/trsc.1090.0301>
- [3] Angelelli, E., Mansini, R.: The vehicle routing problem with time windows and simultaneous pick-up and delivery. In: *Quantitative Approaches to Distribution Logistics and Supply Chain Management*, pp. 249–267. Springer, ??? (2002)
- [4] Wang, J., Zhou, Y., Wang, Y., Zhang, J., Chen, C.L.P., Zheng, Z.: Multiobjective vehicle routing problems with simultaneous delivery and pickup and time windows: Formulation, instances, and algorithms. *IEEE Transactions on Cybernetics* **46**(3), 582–594 (2016) <https://doi.org/10.1109/TCYB.2015.2409837>
- [5] Praxedes, R., Bulhoes, T., Subramanian, A., Uchoa, E.: A unified exact approach for a broad class of vehicle routing problems with simultaneous pickup and delivery. *Computers & Operations Research* **162**, 106467 (2024)
- [6] Wang, H.-F., Chen, Y.-Y.: A genetic algorithm for the simultaneous delivery and pickup problems with time window. *Computers & Industrial Engineering* **62**(1), 84–95 (2012) <https://doi.org/10.1016/j.cie.2011.08.018>
- [7] Wang, C., Mu, D., Zhao, F., Sutherland, J.W.: A parallel simulated annealing method for the vehicle routing problem with simultaneous pickup-delivery and time windows. *Computers & Industrial Engineering* **83**, 111–122 (2015) <https://doi.org/10.1016/j.cie.2015.02.005>
- [8] Hof, J., Schneider, M.: An adaptive large neighborhood search with path relinking for a class of vehicle-routing problems with simultaneous pickup and delivery. *Networks* **74**(3), 207–250 (2019)
- [9] Shi, Y., Zhou, Y., Boudouh, T., Grunder, O.: A lexicographic-based two-stage algorithm for vehicle routing problem with simultaneous pickup-delivery and time window. *Engineering Applications of Artificial Intelligence* **95**, 103901 (2020) <https://doi.org/10.1016/j.engappai.2020.103901>

- [10] Liu, S., Tang, K., Yao, X.: Memetic search for vehicle routing with simultaneous pickup-delivery and time windows. *Swarm and Evolutionary Computation* **66**, 100927 (2021) <https://doi.org/10.1016/j.swevo.2021.100927>
- [11] Lei, Z., Hao, J.-K.: A memetic algorithm for vehicle routing with simultaneous pickup and delivery and time windows. *IEEE Transactions on Evolutionary Computation* **29**(5), 1924–1936 (2025)
- [12] Wu, H., Gao, Y.: An ant colony optimization based on local search for the vehicle routing problem with simultaneous pickup-delivery and time window. *Applied Soft Computing* **139**, 110203 (2023) <https://doi.org/10.1016/j.asoc.2023.110203>
- [13] Liu, F., Wang, L., Gui, M., Zhang, Y., Lan, Y., Lai, C.: A hybrid heuristic algorithm for urban distribution with simultaneous pickup-delivery and time window. *Journal of Heuristics* **29**(2–3), 269–311 (2023)
- [14] Zhang, W., Li, S., Deng, M., Mu, Y., Li, P.: Multi-stage hybrid multiobjective evolutionary algorithm for multi-type vehicle routing problem with simultaneous pickup and delivery and time windows. *Computers & Industrial Engineering* **207**, 111242 (2025)
- [15] Mu, D., Wang, C., Zhao, F., Sutherland, J.W.: Solving vehicle routing problem with simultaneous pickup and delivery using parallel simulated annealing algorithm. *International Journal of Shipping and Transport Logistics* **8**(1), 81–103 (2016)
- [16] Voigt, S.: A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. *European Journal of Operational Research* **322**(2), 357–375 (2025)
- [17] Dhiman, G., Kumar, V.: Spotted hyena optimizer: A novel bio-inspired based metaheuristic technique for engineering applications. *Advances in Engineering Software* **114**, 48–70 (2017) <https://doi.org/10.1016/j.advengsoft.2017.05.014>
- [18] Mirjalili, S., Lewis, A.: The whale optimization algorithm. *Advances in Engineering Software* **95**, 51–67 (2016) <https://doi.org/10.1016/j.advengsoft.2016.01.008>
- [19] Gharehchopogh, F.S., Gholizadeh, H.: A comprehensive survey: Whale optimization algorithm and its applications. *Swarm and Evolutionary Computation* **48**, 1–24 (2019)
- [20] Utama, D.M., Yurifah, A., Garside, A.K.: A novel hybrid spotted hyena optimizer: An algorithm for fuel consumption capacitated vehicle routing problem. *International Journal of Technology* **14**(5), 1049–1060 (2023)
- [21] Zhou, X., Li, J., Xie, F., Fang, J.: Research on origin-based cold storage location

- and routing optimization of fresh agricultural products based on hybrid whale algorithm. *Scientific Reports* **14**, 21078 (2024)
- [22] Nguyen, T., Vaclav, S., Vo, B., Nguyen, V.D.: PH-SHOWOA: Parallel hybrid SHO-WOA for VRPSPDTW. *PLOS ONE* **21**(3), 0343262 (2026) <https://doi.org/10.1371/journal.pone.0343262>
 - [23] Nickolls, J., Buck, I., Garland, M., Skadron, K.: Scalable parallel programming with CUDA. *Queue* **6**(2), 40–53 (2008) <https://doi.org/10.1145/1365490.1365500>
 - [24] Liang, Z., Li, H., Yu, N., Sun, K., Cheng, R.: Bridging evolutionary multiobjective optimization and GPU acceleration via tensorization. *IEEE Transactions on Evolutionary Computation* **30**(1), 420–434 (2026) <https://doi.org/10.1109/TEVC.2025.3555605>
 - [25] Dorneles, L.D.L., Boiani, M., Dorn, M.: Benchmark problems on GPU: Accelerating experiments on metaheuristics. In: 2023 IEEE Congress on Evolutionary Computation, pp. 1–8 (2023)
 - [26] Lin, M., *et al.*: Speeding up local optimization in vehicle routing with tensor-based GPU acceleration. *arXiv preprint arXiv:2506.17357* (2025) <https://doi.org/10.48550/arXiv.2506.17357>
 - [27] Tang, K., Liu, S., Yang, P., Yao, X.: Few-shots parallel algorithm portfolio construction via co-evolution. *IEEE Transactions on Evolutionary Computation* **25**(3), 595–607 (2021)
 - [28] Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983) <https://doi.org/10.1126/science.220.4598.671>
 - [29] Feo, T.A., Resende, M.G.C.: Greedy randomized adaptive search procedures. *Journal of Global Optimization* **6**(2), 109–133 (1995) <https://doi.org/10.1007/BF01096763>